

1.3.6 Train/ Validation/ Test split (stratified 60/20/20)

Trong các bài toán dự đoán, nếu đánh giá mô hình trên chính dữ liệu đã học sẽ dẫn đến đánh giá sai lệch vì mô hình chỉ “học vẹt”. Do đó tập dữ liệu được chia làm 3 phần độc lập:

- Tập huấn luyện (Training Set - 60 %): Môi trường để mô hình cập nhật trọng số và tìm ra các quy luật tương quan ẩn giữa hành vi (VLE, Assessment) và kết quả rủi ro.
- Tập xác thực (Validation Set - 20%): Được sử dụng liên tục trong quá trình tinh chỉnh. Mô hình không học từ tập này nhưng mượn nó để đo lường cấu hình nào đang hoạt động tốt nhất.
- Tập kiểm thử (Test Set - 20%): Tập dữ liệu được “cất kín” chỉ sử dụng một lần duy nhất ở bước cuối cùng để đo lường hiệu năng thực tế của mô hình khi triển khai ngoài hệ thống thực.

Sử dụng kỹ thuật Stratified Split (Phân chia tầng): Dữ liệu rủi ro của sinh viên là không đồng đều. Nếu lấy ngẫu nhiên thông thường, tập Test có thể tình cờ không chứa sinh viên rủi ro High, dẫn đến việc đánh giá mất đi ý nghĩa thực tiễn. Kỹ thuật chia phân tầng đảm bảo rằng dù chia nhỏ ra sao, tỷ lệ phần trăm các lớp Low, Medium, High, Very High ở cả ba tập Train, Valid, Test đều tỉ lệ thuận tuyệt đối với bức tranh toàn cảnh ban đầu

```
# 2. Thực hiện chia tập dữ liệu 60/20/20 với tham số stratify chống suy biến dữ liệu
X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.25, stratify=y_temp, random_state=42)

print(f"\nKích thước tập Train: {X_train.shape}")
print(f"Kích thước tập Validation: {X_val.shape}")
print(f"Kích thước tập Test: {X_test.shape}")
```

1.3.7 Chuẩn hóa dữ liệu (StandardScaler)

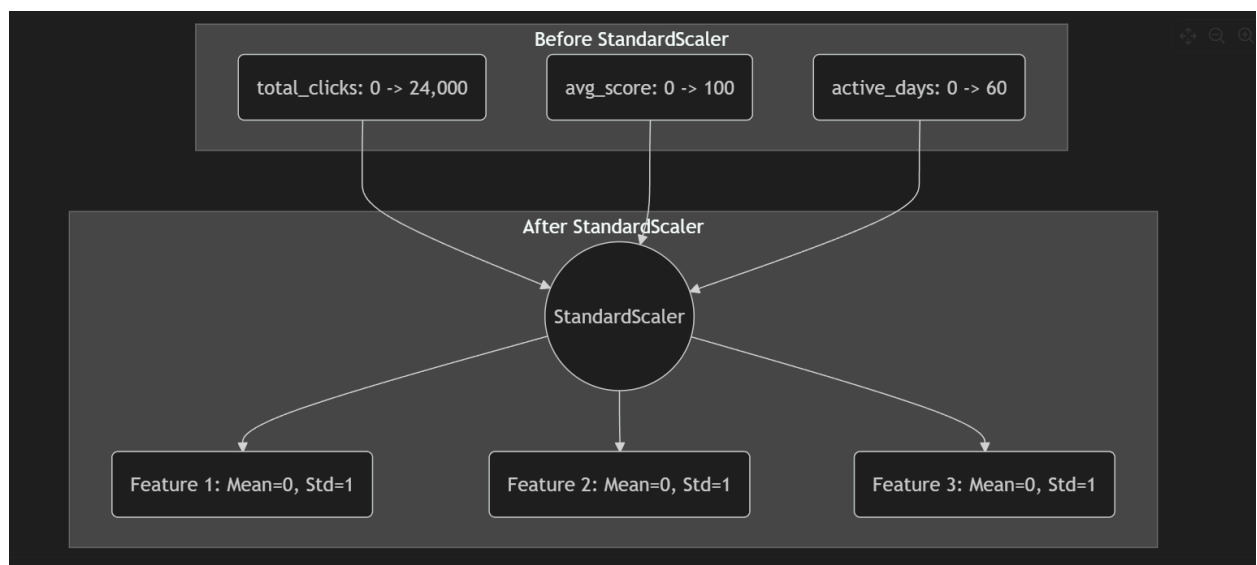
Dữ liệu sinh viên bao gồm các đặc trưng có thang đo (scale) chênh lệch rất lớn. Đặc trưng tần suất tương tác (total_clicks) dao động từ 0 đến hàng chục nghìn lượt, trong khi điểm bình quân (avg_score) chỉ đạt tối đa là 100.

Sự chênh lệch này khiến các thuật toán tối ưu hóa (như Gradient Descent) hoạt động thiếu hiệu quả do bị "thiên vị", nhầm tưởng các biến số có giá trị lớn là quan trọng hơn. Nhóm sử dụng StandardScaler để chuyển dịch toàn bộ dữ liệu về phân phối chuẩn (Standard Normal Distribution):

- Trung bình (μ) = 0
- Độ lệch chuẩn (σ) = 1

Phòng chống rò rỉ dữ liệu (Data Leakage): Scaler chỉ được tính toán trọng số (fit) dựa trên tập Training. Thông số trung bình cục bộ này sau đó được "đóng băng" và chỉ dùng để biến đổi (transform) cho tập Val/Test. Việc này mô phỏng môi trường thực tế khi dự báo luồng sinh viên mới trong tương lai (vốn là những giá trị hoàn toàn ẩn).

```
# 3. Chuẩn hóa dữ liệu StandardScaler và lưu vào Models để triển khai API sau này
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)
joblib.dump(scaler, os.path.join(models_dir, 'scaler.pkl'))
```



Tiến trình đồng nhất hóa thang đo (Scaling) thu hẹp khoảng cách tuyệt đối giữa các biến số

1.4. Phân tích và Xử lý mất cân bằng dữ liệu (Imbalanced Data)

Lý thuyết về mất cân bằng dữ liệu

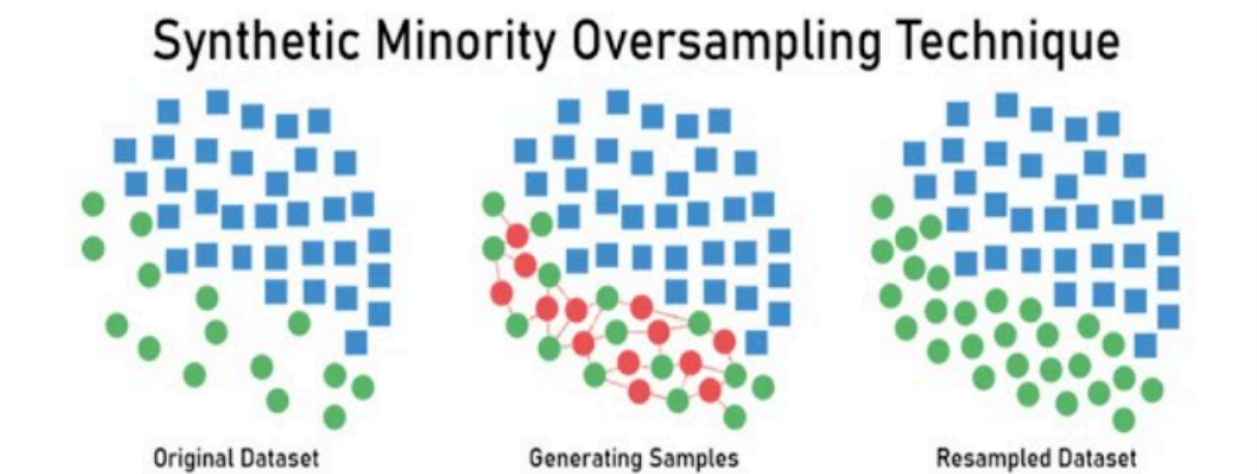
Trong dữ liệu giáo dục, phần đông sinh viên có tình trạng học tập bình thường, trong khi tập sinh viên bỏ học hoặc có rủi ro học vụ cao (High / Very High) chiếm số lượng rất nhỏ.

Sự thiên lệch này gây ảnh hưởng nghiêm trọng đến các mô hình phân loại. Mô hình dễ học được một "mẹo" tiêu cực: chỉ cần dự đoán tất cả học sinh thuộc

nhóm rủi ro "Low" thì độ chính xác (Accuracy) vẫn đạt trên 80%, nhưng hệ thống Cảnh báo Sớm (EWS) sẽ bỏ lọt 100% lứa sinh viên có nguy cơ thực sự. Để giải quyết vấn đề này, dự án áp dụng hai giải pháp song hành:

1. Thuật toán SMOTE (Synthetic Minority Over-sampling Technique): Thay vì sao chép dữ liệu cũ (dễ gây Overfitting), SMOTE sử dụng thuật toán k-Nearest Neighbors. Nó chọn một mẫu thuộc lớp thiểu số, đo các điểm lân cận trong không gian đa chiều, sau đó nội suy toán học để "vẽ" thêm các điểm dữ liệu nhân tạo mới. Điều này làm phong phú tập huấn luyện cho nhóm High và Very High.

SMOTE – Imbalanced Data



2. Trọng số phạt (Class_Weight = 'balanced'): Đây là sự can thiệp trực tiếp vào Hàm mất mát (Loss Function). Nếu mô hình dự đoán nhầm lớp đa số (Low), nó bị trừ điểm ít. Nhưng nếu bỏ lỡ một sinh viên thuộc nhóm Very High, mô hình sẽ bị "trừng phạt" nặng gấp nhiều lần, buộc nó phải học cách nhận diện nhóm này kỹ hơn.

```
# 4. Xử lý mất cân bằng dữ liệu trên tập huấn luyện tạo môi trường công bằng
1:1:1:1
smote = SMOTE(random_state=42)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train_scaled, y_train)

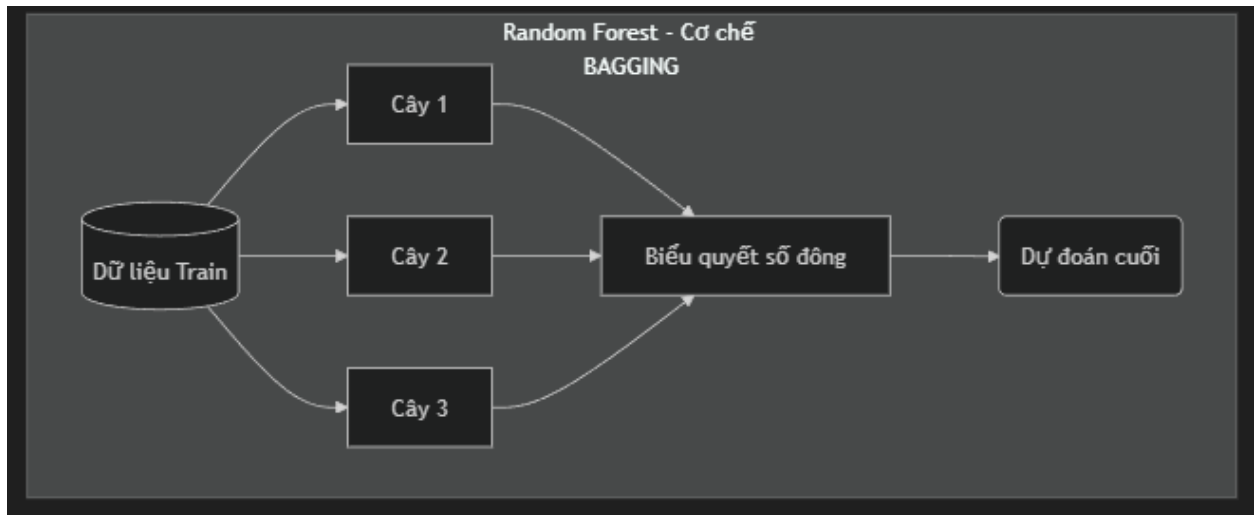
print(f"\nPhân phối nhãn sau thuật toán SMOTE:")
print(pd.Series(y_train_resampled).value_counts())
# Kịch bản Output hoàn hảo: Lớp Low, Medium, High, Very High được cân bằng ngang
bằng nhau
```

1.5. Đào tạo mô hình học máy (Machine Learning Models)

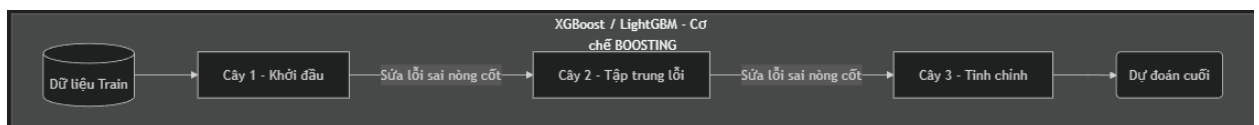
Dự án thực hiện đối sánh quy mô rộng từ kiến trúc tuyến tính cơ sở đến những mạng kết hợp (Ensembles) phức tạp:

```
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42,
class_weight='balanced'),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42,
class_weight='balanced'),
    'XGBoost': XGBClassifier(use_label_encoder=False, eval_metric='mlogloss',
random_state=42),
    'LightGBM': LGBMClassifier(class_weight='balanced', random_state=42)
}
```

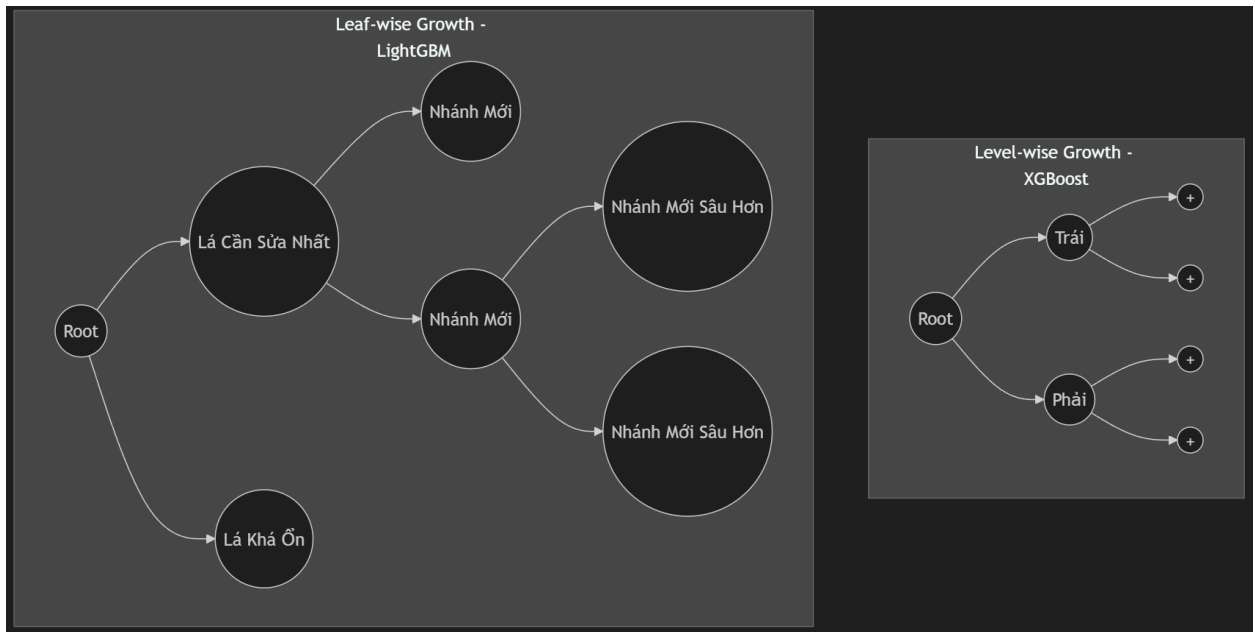
- Logistic Regression (Baseline): Giải quyết bài toán bằng phương trình tuyến tính và hàm kích hoạt Multinomial Softmax. Dù minh bạch nhưng thường yếu thế khi dữ liệu có tương tác đa chiều phức tạp.
- Random Forest: Sử dụng chiến thuật Bagging. Xây dựng độc lập hàng trăm cây quyết định, mỗi cây học một phần dữ liệu và biểu quyết kết quả cuối cùng, giúp triệt tiêu hiện tượng Overfitting.



- XGBoost (Extreme Gradient Boosting): Các cây quyết định được học nối tiếp nhau. Cây sau tập trung sửa lỗi cho cây trước (Minimize Gradient Residuals), là thuật toán thống trị dữ liệu dạng bảng



- LightGBM (Mô hình trọng tâm): Cải tiến từ XGBoost với cơ chế Leaf-wise growth (phát triển cây theo chiều sâu tại lá có lỗi lớn nhất) và sử dụng biểu đồ Histogram để nén dữ liệu. LightGBM cho tốc độ huấn luyện nhanh gấp 5-10 lần và chiếm ít RAM hơn, cực kỳ phù hợp với dữ liệu tương tác VLE không lồ.



1.6. Phương pháp Đánh giá và Cấu trúc Báo cáo Hiệu năng

Bộ chỉ số đo lường được thiết kế đặc thù cho Hệ thống Cảnh báo Sớm (EWS):

1. Macro F1-Score (Chỉ số công bằng): Nhóm chọn Macro F1 làm chỉ số gốc thay vì Accuracy. Macro F1 tính trung bình cộng F1-Score của từng lớp một cách độc lập. Mô hình chỉ đạt điểm cao nếu nhận diện tốt cả những nhóm thiểu số (High/Very High).
2. Ma trận nhầm lẫn (Confusion Matrix): Sử dụng biểu đồ Heatmap để quan sát sự nhầm lẫn giữa các biên giới rủi ro (ví dụ: ranh giới mỏng manh giữa Medium và High). Đường chéo chính biểu thị các trường hợp dự đoán đúng tuyệt đối.
3. AUC-ROC : Với bài toán đa lớp , chiến lược OVR được áp dụng để đo lường diện tích đường cong (AUC) , từ đó làm căn cứ đánh giá khả năng phân cấp ranh giới rủi ro của từng thuật toán.